

Evaluating Communication Topologies in Decentralized Federated Learning Under Heterogeneous Data Distributions

Ashwin Kashyap
Department of Computer Science
Western University
London, Ontario, Canada
akashya6@uwo.ca

Abstract—Decentralized federated learning replaces the central server used in traditional federated learning with direct communication between neighboring clients. This makes communication topology an important part of the training process, particularly when clients have different data distributions. This paper compares five topologies—line, ring, star, hybrid, and mesh—using ten Fashion-MNIST clients. Client data is partitioned with a Dirichlet distribution using $\alpha = 100$, $\alpha = 0.5$, and $\alpha = 0.1$ to create increasing levels of data heterogeneity. In each synchronous communication round, clients train locally for one epoch with Adam and then aggregate model parameters with their neighbors using Metropolis weights. Each configuration is trained for 50 rounds and repeated across ten random seeds. Unweighted and sample-weighted FedAvg baselines are evaluated using the same partitions and training settings. With nearly IID data, final average accuracy ranges from 89.65% to 92.49% across the decentralized topologies. With highly non-IID data, the range increases to 58.00% to 86.34%, while unweighted and sample-weighted FedAvg reach 86.35% and 85.63%, respectively. Mesh remains close to unweighted FedAvg in all settings, whereas the star topology experiences the largest reduction in accuracy. The results show that the communication topology has a greater effect as client data become more heterogeneous. They also support previous findings that the spectral gap describes consensus speed but does not by itself predict empirical deep-learning performance.

Index Terms—decentralized federated learning, communication topology, non-IID data, Dirichlet distribution, Metropolis weights, spectral gap, FedAvg

I. INTRODUCTION

Machine learning is increasingly moving away from settings in which all training data are collected on one machine or in one data center. In many applications, data are produced and retained by hospitals, organizations, and personal devices. Federated learning (FL) allows these participants to train a shared model without first pooling their raw data in one location [1]. Keeping data local reduces the need to move large or sensitive datasets and can lessen privacy and confidentiality concerns, although FL does not guarantee privacy [2].

Compared with traditional distributed optimization, FL must handle two forms of heterogeneity. The first is *systems heterogeneity*: participating devices can differ in computation, memory, network quality, and availability. The second is *statistical*

heterogeneity: local datasets may follow non-identical distributions [7]. This study focuses on statistical heterogeneity, while controlling the systems side through a synchronous simulation.

In centralized federated learning (CFL), a central coordinator distributes the current model, receives locally trained updates, and aggregates them using methods such as Federated Averaging (FedAvg) [1]. This creates a logical star centered on the server. Fully decentralized federated learning (DFL), in contrast, removes the central coordinator and allows clients to exchange models directly with their graph neighbors [3]. The center of a decentralized star, when one is used, is therefore another training client rather than a separate parameter server. More generally, DFL can operate over line, ring, star, hybrid, or mesh topologies. Sparse topologies require fewer communication links per round, but information may need several rounds to reach distant clients.

Natural client datasets are rarely identical. In classification problems, one important form of statistical heterogeneity is *label-distribution skew*, where the relative frequency of classes differs across clients. Local training then moves client models in different directions. Hsieh *et al.* showed that label skew can cause substantial accuracy loss in decentralized learning [4], while D-Cliques showed that the communication topology can be designed to reduce the effect of heterogeneous labels [5]. These findings motivate a controlled comparison of common fixed topologies across increasing levels of non-IID data.

This study investigates the following questions:

- 1) How does communication topology affect convergence and final test accuracy?
- 2) Does the effect of topology increase as client data become more non-IID?
- 3) How does topology affect the difference between average accuracy and the minimum accuracy across client models?
- 4) How closely is spectral gap associated with the observed accuracy of each topology?

The contribution of this work is an empirical comparison of five fixed communication topologies across three levels of client data heterogeneity. The decentralized results are also compared with an unweighted FedAvg baseline and a

sample-weighted FedAvg baseline, using the same model, Dirichlet data partitions, random seeds, and training settings. The source code and result files are available at <https://github.com/ashkash04/dfi-topology-thesis>.

II. RELATED WORK

A. Federated Learning and Non-IID Data

FedAvg introduced communication-efficient federated training through repeated local optimization followed by server-side model averaging [1]. A major challenge is statistical heterogeneity, meaning that client datasets may follow different distributions [2]. Hsu *et al.* used a Dirichlet distribution to generate a continuous range of non-identical class distributions and showed that FedAvg performance decreases as client distributions become more different [6]. FedProx identifies both systems and statistical heterogeneity as central differences between federated learning and conventional distributed optimization, and modifies the local objective to improve stability under heterogeneous conditions [7]. LEAF provides datasets and evaluation tools intended to represent more realistic federated settings, including unequal client sizes and naturally occurring non-IID data [8].

B. Decentralized Learning

Lian *et al.* studied decentralized parallel stochastic gradient descent (D-PSGD), in which workers exchange model parameters with graph neighbors and combine them using a mixing matrix rather than sending updates to a central parameter server [9]. Sun *et al.* proposed decentralized federated averaging, where clients perform multiple local updates before communicating with neighbors in an undirected graph [10]. This is conceptually close to the procedure used in this study. Gossip learning is another server-free approach in which models are exchanged and aggregated directly between nodes [11]. Yuan *et al.* distinguish *continual* DFL, where a model is passed sequentially between clients, from *aggregate* DFL, where a client combines multiple received models before further learning [3]. The method evaluated here belongs to the aggregate paradigm.

C. Communication Topology

Metropolis weights provide a local rule for neighbor averaging. On a connected graph, repeated averaging of fixed node values converges to the network-wide arithmetic mean under the usual assumptions [13]. The topology can affect both learning speed and system throughput [14]. Under heterogeneous data, D-Cliques constructs groups whose combined label distributions are closer to the global distribution, showing that topology can be used to reduce label skew [5].

Vogels *et al.* make a stronger caution: the spectral gap of a communication graph is not, by itself, predictive of empirical performance in deep learning [15]. Spectral gap characterizes repeated averaging, whereas learning continually changes the values being averaged. It should therefore be treated as a descriptor of the mixing process rather than a complete predictor of optimization performance. Dynamic

topology methods provide a further direction by changing peer connections during training instead of using one fixed graph [16].

III. METHODOLOGY

A. Decentralized Training and Mixing

Ten clients participate in each experiment. Every client maintains its own copy of the convolutional neural network. At the beginning of round t , client i holds parameters $\theta_i^{(t)}$. For each local mini-batch $\mathcal{B}_{i,k}$, one Adam update is applied:

$$\theta_{i,0}^{(t)} = \theta_i^{(t)}, \quad (1)$$

$$\theta_{i,k+1}^{(t)} = \text{AdamUpdate}\left(\theta_{i,k}^{(t)}, \mathcal{B}_{i,k}\right), \quad (2)$$

$$\tilde{\theta}_i^{(t)} = \theta_{i,K_i}^{(t)}, \quad (3)$$

where K_i is the number of mini-batches in client i 's one local epoch, and $\tilde{\theta}_i^{(t)}$ denotes its locally trained model before communication.

After every client finishes local training, client i computes

$$\theta_i^{(t+1)} = \sum_{j=1}^N W_{ij} \tilde{\theta}_j^{(t)}, \quad (4)$$

where W_{ij} is the weight assigned by client i to client j . A nonzero off-diagonal entry means that client i must receive client j 's locally trained model. Equation (4) is applied element by element to every convolutional and fully connected parameter in the CNN.

This procedure is inspired by D-PSGD, but it is more accurately described as decentralized federated averaging because each client completes a full local epoch before each communication step [9], [10]. D-PSGD more commonly alternates individual stochastic-gradient steps with frequent mixing, whereas this study performs many Adam updates followed by one model-averaging event.

B. Metropolis Mixing Matrix

Each communication graph $G = (V, E)$ is converted into a symmetric, doubly stochastic mixing matrix W using Metropolis weights [13]. For nodes with degrees d_i and d_j ,

$$W_{ij} = \begin{cases} \frac{1}{1 + \max(d_i, d_j)}, & i \neq j, (i, j) \in E, \\ 1 - \sum_{k \neq i} W_{ik}, & i = j, \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

Each row of W gives one client's averaging rule. Rows and columns sum to one, so a mixing step preserves the arithmetic mean of the locally trained client models.

C. Synchronous Aggregate Procedure

Algorithm 1 summarizes the complete procedure. It is an aggregate DFL algorithm because every client combines its own locally trained model with one or more neighboring models during each round [3]. It is not continual DFL, where

Algorithm 1 Synchronous Aggregate DFL Used in This Study

Require: Graph G , Metropolis matrix W , datasets $\{D_i\}_{i=1}^N$, rounds T

- 1: Initialize identical CNN parameters $\theta_i^{(0)} \leftarrow \theta^{(0)}$ for all clients
- 2: **for** $t = 0$ to $T - 1$ **do**
- 3: **for** each client i (logically in parallel) **do**
- 4: Train $\theta_i^{(t)}$ for one local epoch on D_i using Adam
- 5: Save the locally trained model as $\tilde{\theta}_i^{(t)}$
- 6: **end for**
- 7: Freeze the snapshot $\{\tilde{\theta}_j^{(t)}\}_{j=1}^N$
- 8: **for** each client i (logically in parallel) **do**
- 9: Compute $\theta_{i,\text{new}} \leftarrow \sum_{j=1}^N W_{ij} \tilde{\theta}_j^{(t)}$
- 10: **end for**
- 11: Commit $\theta_i^{(t+1)} \leftarrow \theta_{i,\text{new}}$ for all clients simultaneously
- 12: Every five rounds, evaluate every client model on the common Fashion-MNIST test set
- 13: **end for**

a single model is trained and passed sequentially from client to client.

The weight updates are synchronous. All local models $\{\tilde{\theta}_j^{(t)}\}$ are first saved as one frozen snapshot. Every client then calculates its next model from that same snapshot, and all new models are committed together. No client uses a neighbor's already mixed model from the current round.

D. Communication Topologies and Spectral Gap

Five fixed topologies are evaluated: line, ring, star, hybrid, and mesh, as illustrated in Figure 1. The hybrid graph consists of two rings connected by one bridge edge and is adapted from topology examples discussed by Yuan *et al.* [3].

The spectral gap summarizes how quickly repeated averaging of fixed values approaches consensus. For a symmetric mixing matrix with largest eigenvalue $\lambda_1 = 1$,

$$\text{gap}(W) = \lambda_1 - \max(|\lambda_2(W)|, |\lambda_N(W)|). \quad (6)$$

A larger spectral gap indicates faster consensus under repeated mixing. Spectral gap describes the communication process; it is not itself a measure of classification accuracy.

E. Client Data Partitions

Fashion-MNIST contains 60,000 training images and 10,000 test images from ten clothing classes [17]. To control data heterogeneity, the training images are partitioned using a Dirichlet distribution parameterized by the concentration parameter α .

For each class, a vector of client allocation proportions is drawn from a Dirichlet distribution. Large α values produce more even class allocations, while small values concentrate each class among fewer clients. The experiments use $\alpha = 100$ for a nearly IID setting, $\alpha = 0.5$ for moderate heterogeneity, and $\alpha = 0.1$ for high heterogeneity. This follows the use of the Dirichlet concentration parameter as a continuous control for non-IID data [6]. A heat map of the client distributions for the three settings is shown in Figure 2.

The implementation allocates each class independently and does not force all clients to receive the same total number of samples. Lower α therefore creates both label-distribution skew and quantity skew. Clients with more samples also perform more Adam updates during their one local epoch. These effects are considered when interpreting the results.

F. FedAvg Baseline

The centralized baselines use the same model, client partitions, random seeds, local optimizer, local epochs, and number of rounds as the decentralized experiments. After local training, the unweighted FedAvg baseline computes an unweighted average of all client models:

$$\theta^{(t+1)} = \frac{1}{N} \sum_{i=1}^N \tilde{\theta}_i^{(t)}. \quad (7)$$

The unweighted average is used deliberately because the decentralized Metropolis matrix also gives every client equal weight, making this a matched baseline. Standard FedAvg commonly uses sample weighting when client dataset sizes differ:

$$\theta_{\text{weighted}}^{(t+1)} = \sum_{i=1}^N \frac{n_i}{\sum_{j=1}^N n_j} \tilde{\theta}_i^{(t)}, \quad (8)$$

where $n_i = |D_i|$. This sample-weighted form is also evaluated as a second centralized baseline. It is not a matched baseline for the decentralized methods, which use unweighted mixing rather than sample-count weighting.

IV. EXPERIMENTAL SETUP

The classifier is a small convolutional neural network with two 3×3 convolutional layers containing 32 and 64 channels, max pooling after each convolution, a fully connected layer with 128 units, and a ten-class output layer. The model is deliberately compact because the study focuses on topology and data distribution rather than model architecture.

For each topology, α , and seed, training is run for 50 rounds. This gives $5 \times 3 \times 10 = 150$ decentralized runs. Each FedAvg baseline adds $3 \times 10 = 30$ runs, for 210 training runs in total. All client models begin from the same initialization within a run. The full sweep was run on a laptop with an Intel i9-13900HX, 32 GB of RAM, and an NVIDIA RTX 4090 Laptop GPU.

Every five rounds, every decentralized client model is evaluated on the Fashion-MNIST test set. Two metrics are reported:

- **Average accuracy:** the mean test accuracy across the ten client models.
- **Minimum accuracy:** the lowest test accuracy among the ten client models.

Each FedAvg baseline produces one global model, so only its global test accuracy is reported.

V. RESULTS

A. Convergence

Figure 3 compares average test accuracy at five-round intervals over 50 communication rounds. With $\alpha = 100$, the methods converge to relatively similar accuracies, although the star remains lower than the other topologies. At $\alpha = 0.5$, mesh and both FedAvg baselines separate more clearly from the sparse topologies. At $\alpha = 0.1$, the effect of topology is much larger: mesh and both FedAvg baselines remain above 85%, while the star finishes below 60% on average.

TABLE I
TOPOLOGY AND PER-ROUND COMMUNICATION PROPERTIES FOR TEN CLIENTS

Topology	Edges $ E $	Degree range	Model sends/round $2 E $	Spectral gap
Line	9	1–2	18	0.0326
Ring	10	2	20	0.1273
Star	9	1–9	18	0.1000
Hybrid	11	2–3	22	0.0538
Mesh	45	9	90	1.0000

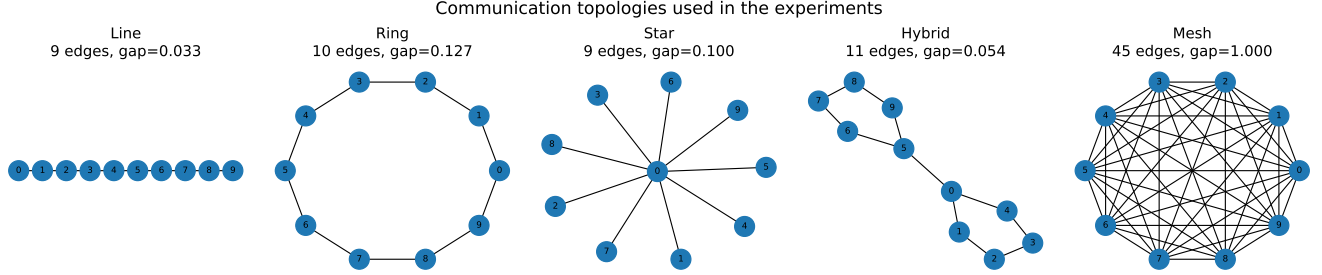


Fig. 1. Communication topologies used in the experiments. Each panel shows the ten-client graph, number of edges, and spectral gap.

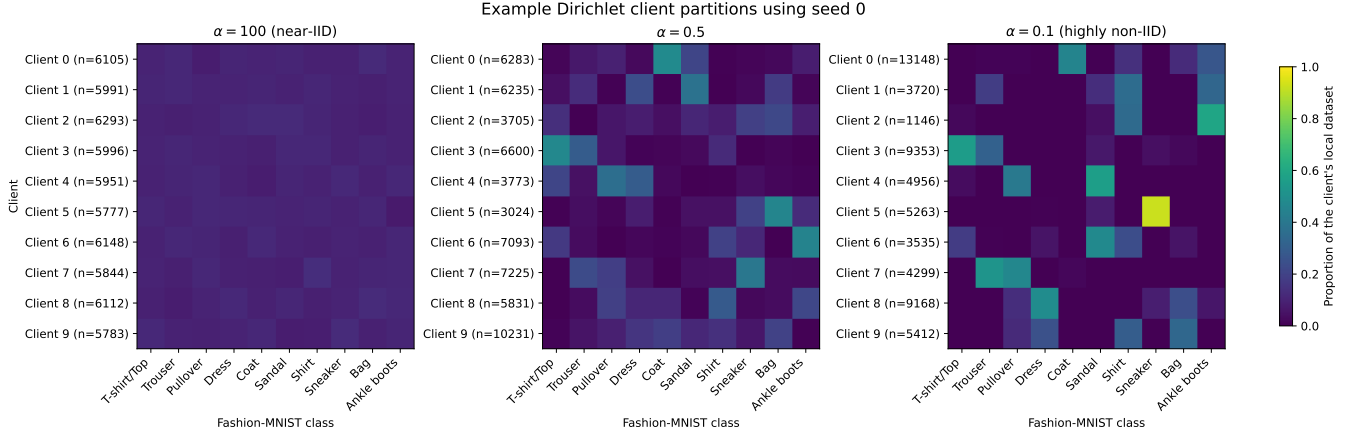


Fig. 2. Example client class distributions for seed 0. Lower α produces more uneven class distributions and greater differences in total client dataset size.

TABLE II
EXPERIMENTAL CONFIGURATION

Setting	Value
Dataset	Fashion-MNIST
Clients	10
Communication rounds	50
Local epochs per round	1
Batch size	64
Optimizer	Adam
Learning rate	10^{-3}
Dirichlet α	100, 0.5, 0.1
Random seeds	0–9
Topologies	Line, ring, star, hybrid, mesh
Baselines	unweighted and sample-weighted FedAvg

B. Final Accuracy

Table III reports the final accuracy after round 50. With nearly IID ($\alpha = 100$) data, the difference between the best

and worst decentralized topology is 2.84 percentage points. With highly non-IID ($\alpha = 0.1$) data, the difference increases to 28.34 points. Mesh loses 6.15 points between $\alpha = 100$ and $\alpha = 0.1$, while the star loses 31.65 points.

Mesh closely matches unweighted FedAvg at all three heterogeneity levels. Their final accuracies differ by only 0.01 points at $\alpha = 100$, 0.04 points at $\alpha = 0.5$, and 0.01 points at $\alpha = 0.1$. This is expected because every row of the mesh Metropolis matrix assigns equal weight to all clients, so all client models receive the same unweighted average after each communication round. Sample-weighted FedAvg is also close, but its final mean accuracy is lower than unweighted FedAvg by 0.05, 0.18, and 0.73 points at $\alpha = 100$, $\alpha = 0.5$, and $\alpha = 0.1$, respectively.

C. Average and Minimum Accuracy

The difference between average and minimum accuracy is small with nearly IID data but becomes much larger as

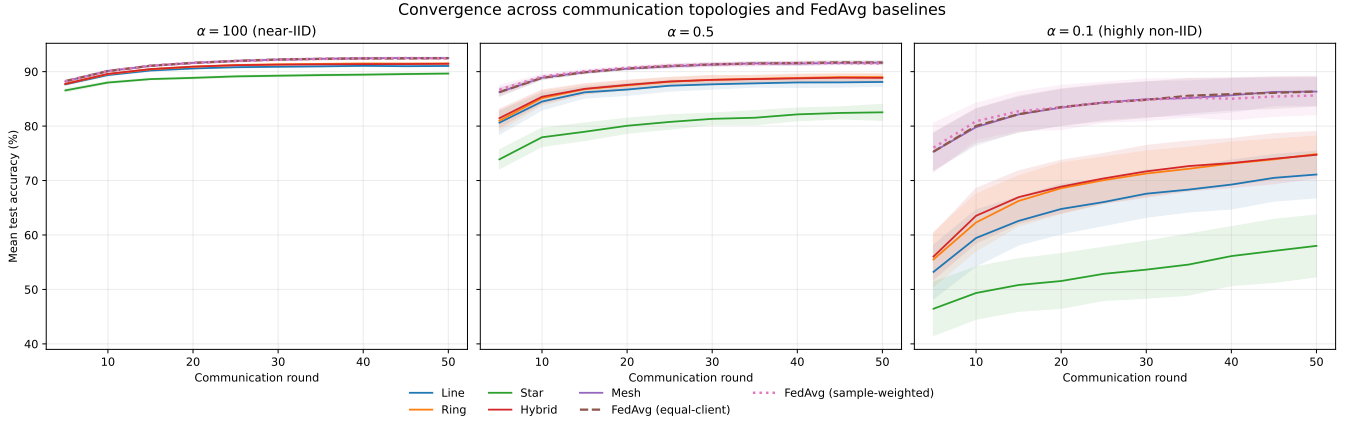


Fig. 3. Average test accuracy at five-round intervals over 50 communication rounds for the five decentralized topologies and two FedAvg baselines. Curves show the mean across ten seeds, and shaded regions show one standard deviation.

TABLE III
ROUND-50 TEST ACCURACY IN PERCENT (MEAN $\pm$ STANDARD DEVIATION ACROSS TEN SEEDS)

Method	$\alpha = 100$	$\alpha = 0.5$	$\alpha = 0.1$
Line	91.04 ± 0.08	88.11 ± 0.78	71.11 ± 4.29
Ring	91.50 ± 0.12	89.00 ± 0.61	74.87 ± 3.32
Star	89.65 ± 0.12	82.53 ± 1.47	58.00 ± 5.68
Hybrid	91.44 ± 0.12	88.86 ± 0.68	74.73 ± 4.28
Mesh	92.49 ± 0.15	91.64 ± 0.26	86.34 ± 2.54
FedAvg (unweighted)	92.48 ± 0.17	91.68 ± 0.30	86.35 ± 2.67
FedAvg (sample-weighted)	92.42 ± 0.17	91.50 ± 0.28	85.63 ± 3.52

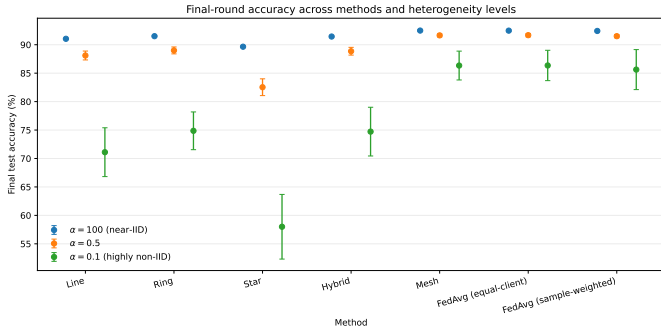


Fig. 4. Round-50 test accuracy across the five decentralized topologies and two FedAvg baselines. Markers show means across ten seeds, and error bars show one standard deviation.

heterogeneity increases. At $\alpha = 0.1$, the final average-to-minimum gaps are 15.25 points for the line, 9.71 for the ring, 18.38 for the star, and 9.48 for the hybrid. Mesh has a gap of zero because all client models are fully connected and identical after averaging.

These results show that topology affects not only the network average but also how differently the individual client models perform.

D. Spectral Gap and Accuracy

Figure 6 compares each topology’s spectral gap with its final average accuracy. Mesh has both the largest spectral gap and

the highest decentralized accuracy, and the ring has a larger gap and higher accuracy than the line. These examples show that spectral gap can still be related to performance.

However, the ordering is not consistent. The star has a spectral gap of 0.1000, larger than the gaps of both the line and hybrid, yet it has the lowest accuracy in all three data settings. At $\alpha = 0.1$, the star reaches 58.00%, compared with 71.11% for the line and 74.73% for the hybrid. The star is therefore a concrete counterexample to treating a larger spectral gap as a guarantee of higher learning accuracy.

This result directly supports the caution of Vogels *et al.* that spectral gap is not predictive of empirical deep-learning performance by itself [15]. In pure consensus, the same fixed values are repeatedly multiplied by W , allowing the network to approach agreement smoothly. In this experiment, each client first performs an entire local epoch, changing thousands of CNN parameters according to its current non-IID data. The next mixing step therefore acts on a newly calculated set of parameters rather than continuing consensus on a fixed state. This effect becomes stronger as α decreases and local updates become more different. Spectral gap remains useful for describing the averaging operator, but it does not capture the directions or magnitudes of the local learning updates, the unequal roles of graph nodes, or the assignment of datasets to those nodes.

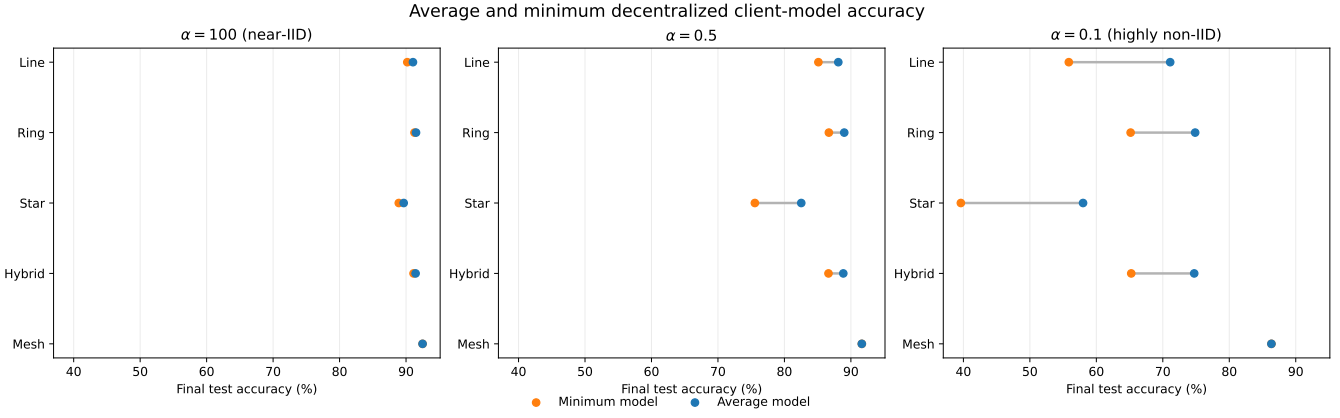


Fig. 5. Final average accuracy and minimum accuracy across decentralized client models, averaged across ten seeds. Larger gaps indicate greater differences in model performance across the network.

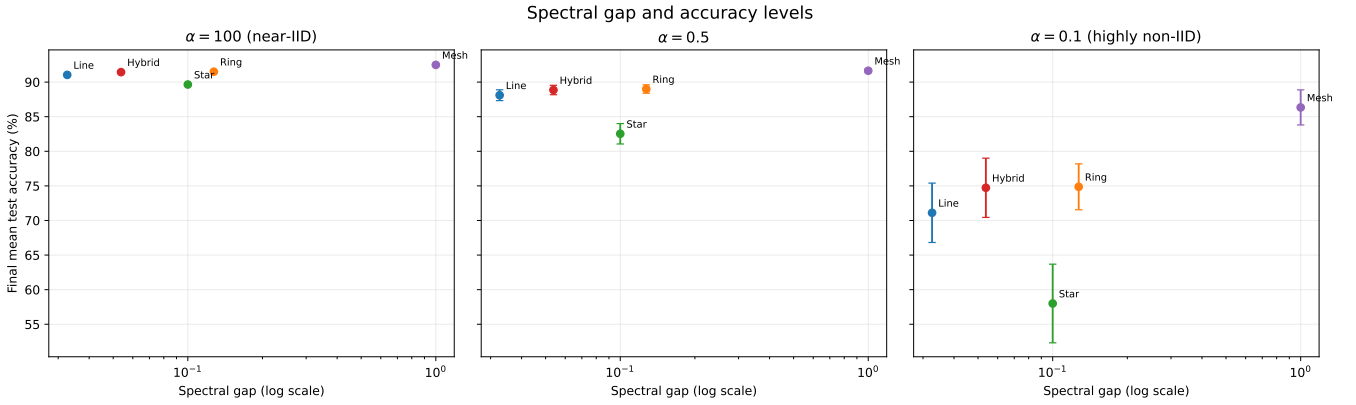


Fig. 6. Spectral gap versus final average test accuracy for the five decentralized topologies. Each point shows the mean across ten seeds, and error bars show one standard deviation.

VI. DISCUSSION

A. Effect of Data Heterogeneity

The main pattern is that topology matters more when client data are more heterogeneous. With $\alpha = 100$, clients receive similar class distributions, so their local models remain relatively similar and sparse communication causes only a small loss. As α decreases, clients train on increasingly different data and also hold different numbers of samples. Their local models therefore differ more after each local epoch, making communication more important.

Mesh performs well because every client receives the network-wide average after each round. Sparse graphs require updates to pass through intermediate clients before reaching the rest of the network. This pattern is consistent with prior findings that label skew harms decentralized training [4] and that topology can be designed to compensate for heterogeneous data [5].

B. Star Topology

The star is the clearest example of why graph connectivity cannot be summarized by spectral gap alone. With the implemented Metropolis weights, every leaf keeps 90% of its own

locally trained model and receives 10% from the hub. The hub, in contrast, averages its own model with all nine leaves using a weight of 0.1 for each. The communication roles of the hub and leaves are therefore very different even though the graph has a larger spectral gap than the line and hybrid.

The result may also depend on which client dataset is assigned to the hub. Under small α , a client can receive an unusually large, small, or skewed dataset. Client datasets remain assigned to the same graph nodes within each run. Consequently, the results may partly depend on which dataset is assigned to an important position, such as the hub of the star or the bridge of the hybrid graph. Although the star consistently performs poorly in the tested settings, the present experiments do not determine whether this is caused only by topology or by an interaction between topology and dataset placement.

C. Accuracy and Communication Cost

Mesh achieves the highest decentralized accuracy per communication round, but it also contains 45 edges and requires 90 directed model sends per round, compared with 18–22 for the sparse topologies. Over 50 rounds, the total numbers of

directed model sends are 900 sends for line and star, 1,000 for ring, 1,100 for hybrid, and 4,500 for mesh. If the centralized baseline is counted as ten uploads and ten downloads per round, it requires 20 directed model transmissions per round.

The study does not measure model size in bytes, latency, bandwidth, or wall-clock network time, so it does not establish that mesh is the most communication-efficient option. Network topology involves a trade-off between model performance and communication cost [5], [14]. Future communication-aware analysis should plot accuracy against cumulative model sends or transmitted bytes rather than rounds alone.

VII. LIMITATIONS

The results should be interpreted within the scope of the experiment.

Training method: The implementation is inspired by D-PSGD but is closer to decentralized FedAvg because it uses one complete local Adam epoch between communication rounds. A new Adam optimizer is created for every local training call, so optimizer state is not carried between rounds.

Data partitioning: The class-wise Dirichlet allocation creates both label-distribution differences and unequal client dataset sizes. The experiment therefore does not isolate label skew from quantity skew. Hsu *et al.* used balanced client populations in their Dirichlet experiments, whereas the implementation used here does not enforce equal client sizes [6].

FedAvg aggregation: The matched FedAvg baseline uses an unweighted average to match the decentralized topologies, while the additional sample-weighted baseline weights client models by local sample count [1]. The decentralized methods remain based on doubly stochastic unweighted Metropolis mixing, so the study does not evaluate decentralized methods designed for sample-weighted aggregation.

Experimental scale: The study uses one dataset, one compact CNN, ten clients, five static topologies, ten random seeds, and 50 rounds. The larger standard deviations at $\alpha = 0.1$ indicate greater run-to-run variation under severe heterogeneity. Additional datasets, models, client counts, and training durations are needed for broader conclusions.

Assignment of data to nodes: Client datasets are not randomly permuted across graph positions. Results for hubs, endpoints, and bridge nodes may therefore depend partly on the particular dataset assigned to those positions.

Network simulation: Communication is synchronous and has no simulated latency, bandwidth limit, client dropout, or device variability. Real systems also exhibit systems heterogeneity [7], [8].

VIII. FUTURE WORK

A direct extension would use balanced Dirichlet partitions so that the class distribution changes while each client keeps the same number of samples. Randomly permuting datasets across graph positions would help separate topology effects from the effect of assigning a particular partition to the hub, endpoints, or bridge nodes.

Future experiments should also include additional datasets such as CIFAR-10 and FEMNIST, more clients, longer training durations, and different model architectures. Communication-aware evaluation should compare accuracy against cumulative model sends, transmitted bytes, and wall-clock time. Finally, adaptive topology methods could be compared with fixed graphs. D-Cliques groups clients according to label distributions [5], while dynamic approaches change peer selection during training [16]. These directions reflect the broader observation that no single topology is best for every decentralized learning setting [12].

IX. CONCLUSION

This paper compared five communication topologies for synchronous aggregate decentralized federated learning under three levels of Fashion-MNIST data heterogeneity. Topology had a relatively small effect with nearly IID data and a much larger effect with highly non-IID data. Mesh remained close to the matched unweighted FedAvg baseline, while sample-weighted FedAvg produced similar centralized accuracy and the star experienced the largest reduction in accuracy. Sparse topologies also showed larger differences between average accuracy and minimum accuracy across client models.

The results provide empirical support, within this experimental setting, for the claim that spectral gap alone does not predict deep-learning performance. The star had a larger spectral gap than the line and hybrid but substantially lower accuracy. Because local training changes the model parameters before every mixing step, the network never performs uninterrupted consensus on a fixed state. Accuracy instead depends on the interaction among graph structure, Metropolis weights, local training, data heterogeneity, and dataset placement. These conclusions remain limited by the use of one dataset, one model, ten clients, fixed graph positions, and communication rounds that are not normalized by cost.

ACKNOWLEDGMENT

The author thanks Dr. Zubair Md Fadlullah for his supervision and feedback throughout this undergraduate thesis study.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics*, vol. 54, 2017, pp. 1273–1282.
- [2] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [3] L. Yuan, Z. Wang, L. Sun, P. S. Yu, and C. G. Brinton, "Decentralized federated learning: A survey and perspective," *IEEE Internet of Things Journal*, vol. 11, no. 21, pp. 34617–34638, 2024.
- [4] K. Hsieh, A. Phanishayee, O. Mutlu, and P. B. Gibbons, "The non-IID data quagmire of decentralized machine learning," in *Proc. 37th Int. Conf. Machine Learning*, vol. 119, 2020, pp. 4387–4398.
- [5] A. Bellet, A.-M. Kermarrec, and E. Lavoie, "D-Cliques: Compensating for data heterogeneity with topology in decentralized federated learning," arXiv:2104.07365, 2021.
- [6] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," arXiv:1909.06335, 2019.

- [7] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Machine Learning and Systems*, vol. 2, 2020.
- [8] S. Caldas, S. M. K. Duddu, P. Wu, T. Li, J. Konečný, H. B. McMahan, V. Smith, and A. Talwalkar, "LEAF: A benchmark for federated settings," arXiv:1812.01097, 2019.
- [9] X. Lian, C. Zhang, H. Zhang, C.-J. Hsieh, W. Zhang, and J. Liu, "Can decentralized algorithms outperform centralized algorithms? A case study for decentralized parallel stochastic gradient descent," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5331–5341.
- [10] T. Sun, D. Li, and B. Wang, "Decentralized federated averaging," arXiv:2104.11375, 2021.
- [11] I. Hegedűs, G. Danner, and M. Jelasity, "Gossip learning as a decentralized alternative to federated learning," in *Distributed Applications and Interoperable Systems*. Springer, 2019, pp. 74–90.
- [12] W.-C. Chung, C.-A. Lo, Y.-H. Lin, Z.-H. Chen, and C.-L. Hung, "Decentralized federated learning with non-IID data: Challenges, trends, and future opportunities," *ACM Computing Surveys*, vol. 58, no. 8, pp. 192:1–192:41, 2026.
- [13] L. Xiao, S. Boyd, and S. Lall, "Distributed average consensus with time-varying Metropolis weights," unpublished manuscript, Jun. 2006.
- [14] G. Neglia, G. Calbi, D. Towsley, and G. Vardoyan, "The role of network topology for distributed machine learning," in *Proc. IEEE INFOCOM*, 2019, pp. 2350–2358.
- [15] T. Vogels, H. Hendrikx, and M. Jaggi, "Beyond spectral gap: The role of the topology in decentralized learning," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [16] A. Ioannou, "Dynamic topology optimization for non-IID data in decentralized learning," Bachelor's thesis, Delft University of Technology, 2025.
- [17] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," arXiv:1708.07747, 2017.